DAA Assignment 2

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 2 – CO2

CO2 Coverage: Brute-Force and Exhaustive-Search techniques – Selection Sort, Bubble Sort, Insertion Sort, Sequential Search, Brute-Force String Matching, Closest Pair, Convex Hull and Exhaustive Search.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: 22/10/26

Name: M Sivam Kumar (192521017)

Q1 – Sorting Under Different Data Conditions **6 Marks**

Problem Context – E-Commerce Order Processing

An e-commerce fulfilment centre arranges customer orders according to a **dispatch priority score**.

The development team is considering:

Selection Sort | Bubble Sort | Insertion Sort

However, the incoming data may be randomly ordered, nearly sorted, already sorted or significantly disordered.

Your Challenge

Investigate whether one of the three algorithms should always be preferred or whether the appropriate choice depends on the **characteristics of the input data**.

Before implementation:

- Predict how the three algorithms will behave under different input arrangements.
- Explain the algorithmic reason for your prediction.

Design **two deliberately contrasting datasets** and **one additional edge-case dataset of your own** that will expose meaningful behavioural differences among the algorithms.

Implement the three algorithms and record appropriate measures such as:

- Number of comparisons

- Number of swaps/movements
- Execution time, where useful

Compare your prediction with the observed behaviour.

Design Decision

The fulfilment centre reports that approximately **95% of existing orders are already in the correct priority order**, with only small batches of new orders added periodically.

Determine which of the three algorithms you would recommend and defend your decision.

Evidence Expected

Reasoned prediction, three deliberately designed input conditions, implementation of the three sorting algorithms, operation counts, comparison of predicted and observed behaviour, and an evidence-based recommendation.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Selection, Bubble & Insertion Sort	**6**
Q2	Sequential Search	**4**
Q3	Brute-Force String Matching	**6**
Q4	Brute-Force Closest Pair	**6**
Q5	Brute-Force Convex Hull	**7**
Q6	Exhaustive Search	**8**
Q7	**Day-4 Design Challenge – Diagnose, Redesign & Validate**	**13**
	TOTAL	**50**

Submission Requirements

Submit:

- Your assignment report containing the required evidence for Q1–Q7
- Source code for all implemented solutions
- Raw experimental data used to produce your results
- Tables/graphs used in your analysis, where applicable

Operation counts, comparison counts or other algorithm-specific measures should be treated as **primary experimental evidence**. Execution time may be reported as additional evidence but should not be used as the sole basis for your conclusions.

Assessment Verification

Students may be asked to explain any part of their:

- Algorithmic design
- Source code
- Test-data design
- Experimental results
- Predictions
- Analysis
- Design Challenge

during a brief verification discussion/viva.

The student should therefore be able to **explain and defend the evidence submitted as their own work**.

Overall Assessment Expectation

This assignment does not primarily assess your ability to reproduce textbook algorithms.

It assesses your ability to:

Predict → Design → Implement → Measure → Analyse → Evaluate → Diagnose → Redesign → Validate

Your conclusions and design decisions must be supported by **your own mathematical reasoning and experimental evidence**.

Answer – Q1: Sorting Under Different Data Conditions

1. Prediction (made before implementation)

Notation: n = number of orders, C = key comparisons, M = data movements (swaps for Selection/Bubble, element shifts for Insertion). Bubble Sort is the standard version with an early-exit flag.

Input	Selection Sort	Bubble Sort	Insertion Sort
Already sorted	$C = n(n-1)/2$, $M = 0$	$C = n-1$, $M = 0$ (one pass)	$C = n-1$, $M = 0$
Reverse order	$C = n(n-1)/2$, $M \approx n/2$	$C = M = n(n-1)/2$	$C = M = n(n-1)/2$
Random	$C = n(n-1)/2$, $M \leq n-1$	$C \approx n^2/2$, $M \approx n^2/4$	$C \approx M \approx n^2/4$
95% sorted + few new items at the end	$C = n(n-1)/2$ (unchanged)	C close to $n^2/2$ (many passes still needed)	$C \approx n$ + inversions (small)

Algorithmic reasons. Selection Sort always scans the whole unsorted part to find the minimum, so its comparisons do not depend on the data. Bubble Sort stops early only if a complete pass makes no swap; a small element placed at the end moves left only one position per pass, so a few late arrivals still force many passes. Insertion Sort shifts an element only past larger elements, so its total shifts equal the number of inversions in the input and it is fast when few inversions exist.

2. Test data designed (n = 1000 distinct priority scores)

- **Dataset A – Sorted** (best-case contrast).
- **Dataset B – Reverse sorted** (worst-case contrast, maximum inversions).
- **Dataset C – Edge case, 95% sorted + 5% new orders:** 950 orders already in priority order followed by 50 random new orders appended (this mirrors the fulfilment-centre report).
- Extra checks: Random order, and All-equal scores (duplicates).

3. Implementation (core of the three algorithms)

```
def selection(a):
    for i in range(n-1):
        k = i
        for j in range(i+1, n):
            C += 1
            if a[j] < a[k]: k = j
        if k != i: swap(a[i], a[k]); M += 1
```

```
def bubble(a):
    for i in range(n-1):
        swapped = False
        for j in range(n-1-i):
            C += 1
            if a[j] > a[j+1]: swap; M += 1; swapped = True
        if not swapped: break

def insertion(a):
    for i in range(1, n):
        v = a[i]; j = i-1
        while j >= 0:
            C += 1
            if a[j] > v: a[j+1] = a[j]; M += 1; j -= 1
            else: break
        a[j+1] = v
```

4. Observed results (C = comparisons, M = movements, time in ms, mean of 3 runs)

Dataset	Selection C / M / ms	Bubble C / M / ms	Insertion C / M / ms
A – Sorted	499,500 / 0 / 46.53	999 / 0 / 0.08	999 / 0 / 0.12
B – Reverse	499,500 / 500 / 37.77	499,500 / 499,500 / 89.62	499,500 / 499,500 / 75.39
C – 95% sorted + 5% new	499,500 / 946 / 34.34	497,789 / 21,852 / 41.72	22,851 / 21,852 / 3.15
Random	499,500 / 990 / 34.51	497,789 / 250,110 / 68.25	251,107 / 250,110 / 33.95
All equal	499,500 / 0 / 34.41	999 / 0 / 0.08	999 / 0 / 0.13

5. Prediction versus observation

- Sorted: Bubble and Insertion needed only 999 = n−1 comparisons and 0 movements, while Selection still used 499,500 comparisons – exactly as predicted.
- Reverse: all three hit their worst case. Bubble and Insertion each made 499,500 comparisons and 499,500 movements; Selection made only 500 swaps.

- Random: Insertion made 251,107 comparisons and 250,110 shifts (close to $n^2/4 \approx 249{,}750$); Bubble made 497,789 comparisons, so about twice the work of Insertion.
- 95% sorted: Insertion needed 22,851 comparisons and 21,852 shifts, whereas Bubble still needed 497,789 comparisons (no real benefit from early exit) and Selection 499,500. This confirmed the prediction that Bubble Sort is not helped by a few late arrivals. The number of Insertion shifts equals the number of inversions in the data.
- Time (ms) followed the same ordering: Insertion 3.15 ms versus Bubble 41.72 ms and Selection 34.34 ms on dataset C.

6. Design decision and recommendation

No single algorithm is best for all inputs – the best choice depends on how disordered the data is. For the fulfilment centre, where about 95% of the orders are already in priority order, I recommend **Insertion Sort**.

- It needed about 22 times fewer comparisons (22,851 versus 497,789 and 499,500) and about 11–13 times less time than the other two on the 95%-sorted data.
- Its cost is proportional to n plus the number of out-of-place pairs, so it adapts automatically to nearly-sorted batches.
- It is stable (orders with equal priority keep their arrival order), which matters for fair dispatch; Selection Sort is not stable.
- Selection Sort would be preferable only if writing to memory is very expensive (it makes the fewest swaps, 946 versus 21,852), which is not the case here. Bubble Sort has no scenario in which it beats Insertion Sort in these tests.
- If the number of new orders grows large (data becomes random-like), Insertion Sort degrades to about $n^2/4$ operations and a different method would have to be considered.

Q2 – When Is Sequential Search an Appropriate Engineering Choice? 4 Marks

Problem Context – Aircraft Maintenance Tool Tracking

An aircraft maintenance facility maintains a temporary list of specialised tools currently issued to technicians.

Each record contains:

Tool ID | Technician ID | Issue Time | Work Bay

The list changes continuously as tools are issued and returned.

When a technician asks:

> "Who currently has Tool T-184?"

the system uses **Sequential Search**.

Your Challenge

Design experiments to investigate Sequential Search under:

- Successful searches at different positions
- Unsuccessful searches
- Different list sizes

Record the **number of comparisons** required.

Then answer:

> **Under what operating conditions is Sequential Search a reasonable engineering choice, and when would you stop recommending it?**

Consider both:

- Dataset size
- Frequency of search requests

Changed Condition

The organisation expands from one hangar to **20 maintenance facilities**, resulting in substantially more active records and lookup requests.

Re-evaluate your original recommendation.

Evidence Expected

Designed search experiments, comparison-count evidence, analysis of successful and unsuccessful searches, an evidence-based operating boundary and a justified response to the changed operating condition.

Answer – Q2: Sequential Search for Tool Tracking

1. Experiment design

Lists of n = 100, 1,000, 10,000 and 100,000 tool records were created (Tool IDs T-0 … T-(n−1)). For each size, I searched for a tool at the first position, the middle, the last position and a Tool ID that is not in the list (unsuccessful search). The number of key comparisons was counted as primary evidence and time recorded as additional evidence.

```
def seq_search(L, key):
    comparisons = 0
    for i in range(len(L)):
        comparisons += 1
        if L[i] == key: return i, comparisons
    return -1, comparisons
```

2. Results (comparisons; time in ms in brackets)

n	First	Middle	Last	Unsuccessful	Average of random successful
100	1 (0.0)	51 (0.003)	100 (0.005)	100 (0.004)	50.5
1,000	1 (0.001)	501 (0.03)	1,000 (0.068)	1,000 (0.063)	500.5
10,000	1 (0.0)	5,001 (0.365)	10,000 (0.751)	10,000 (0.718)	4962.7
100,000	1 (0.001)	50,001 (3.827)	100,000 (7.808)	100,000 (7.445)	50956.1

3. Analysis

- Successful search at position i costs exactly i comparisons: best case 1, worst case n, average about (n+1)/2 (measured averages 50.5, 500.5, 4,962.7 and 50,956.1 for random targets).
- An unsuccessful search always costs n comparisons, because every record must be examined before the tool can be declared “not issued”.
- The cost grows linearly with n: multiplying the list size by 10 multiplies the comparisons and the time by about 10 (for the last position, 0.75 ms at 10,000 records and 7.8 ms at 100,000 records in this environment, roughly 75 ns per comparison).

4. Operating boundary

Total search load is (requests per second λ) × (average comparisons ≈ n/2). With 75 ns per comparison, keeping the search work below 10% of one processor core gives the rule of thumb **$\lambda \times n \lesssim 2.5$ million** (for example, n = 1,000 with up to about 2,500 requests/s, or n = 10,000 with up to about 250 requests/s). I also require a worst-case single lookup below 10 ms, which holds up to n ≈ 100,000 here.

- **Reasonable choice:** small or medium lists (up to a few thousand records), modest request rates, and a list that changes continuously (issue = append, return = remove), because no index has to be maintained, there is no preprocessing, and the code is simple and obviously correct.
- **Stop recommending it when:** λ × n exceeds about 10^6, when n grows beyond about 10^4–10^5 with frequent look-ups, or when unsuccessful or worst-case searches become frequent and a latency target is missed.

Assumption: a single hangar has roughly 500 tools on issue and a few queries per minute. Then λ × n is only a few thousand and Sequential Search is a sound engineering choice.

5. Changed condition – 20 maintenance facilities

Active records become about 20 × 500 = 10,000 and requests about 20 times higher (for example 20 requests per second). Then λ × n ≈ 200,000, which is still below my limit, and a worst-case lookup costs about 0.75 ms, so a single list would still work – but the safety margin is now only about 12 times and every unsuccessful lookup scans all 10,000 records. My revised recommendation is therefore:

- Keep Sequential Search **inside each facility's list** (n ≈ 500) when the facility is known – this keeps the original design valid.
- For a company-wide "Who has Tool T-184?" query, use a **hash table keyed by Tool ID** (average O(1) insert, remove and lookup). In my measurements a hash lookup took about 0.0001 ms independent of n, against 7.8 ms for the full scan at n = 100,000.
- A sorted list with binary search is not suitable here, because each issue/return would require O(n) shifting to keep the order.
- Switch from Sequential Search to the hash table once n passes about 10,000 or λ × n passes about 10^6.

Q3 – Engineer the Behaviour of Brute-Force String Matching 6 Marks

Problem Context – Cybersecurity Log Screening

A security operations centre searches large text-based system logs for known event signatures represented as character patterns.

The initial system uses **Brute-Force String Matching**.

Your Challenge

Instead of merely implementing the algorithm, investigate **how the structure of the text and pattern influences computational effort**.

Design:

1. One text-pattern pair in which the algorithm performs relatively little work.
2. One text-pattern pair deliberately engineered to cause a **large number of character comparisons**.
3. One additional test condition of your own choice.

Before executing each test:

- Predict the expected behaviour.
- Explain the algorithmic reason for your prediction.

Implement the algorithm and record the number of character comparisons.

Inquiry

Compare your prediction with the experimental result.

Explain:

- Why one input requires substantially more work than another
- Which characteristics of the text and pattern create repeated comparisons
- What this reveals about evaluating an algorithm using only its input size

Evidence Expected

Correct implementation, three deliberately designed test conditions, prediction before execution, character-comparison evidence, and a reasoned explanation of the observed differences.

Answer – Q3: Brute-Force String Matching

1. Algorithm

```
def brute_force(text, pat):
    n, m = len(text), len(pat); C = 0
    for i in range(n-m+1):          # every alignment
        j = 0
        while j < m:
            C += 1                   # one character comparison
            if text[i+j] != pat[j]: break
            j += 1
        if j == m: report match at i
```

2. Test conditions, predictions and observations (text length n = 10,000)

Test	Text and pattern	Prediction	Observed comparisons
A – little work	Random lowercase text without the letter z; pattern zzzzzzzz (m = 8)	Every alignment fails at the first character: n−m+1 = 9,993 comparisons	9,993
B – engineered worst case	Text = 10,000 × “A”; pattern AAAAAAAB (m = 8)	Every alignment matches 7 characters and fails on the 8th: (n−m+1) × m = 79,944	79,944
C – own test: realistic log	Log-like lines (INFO, GET, 404 …); pattern “ERROR 500” occurring once (m = 9)	Mostly first-character failures, so close to n, plus a few extra for the match; about 10,000–10,200	10,131 (1 match)
D – extra: binary text	Random 0/1 text; pattern 00000001	Partial matches average about 2 characters, so about 2n = 20,000	19,653 (42 matches)

Effect of pattern length on test B (n = 10,000): m = 4 → 39,988; m = 8 → 79,944; m = 16 → 159,760; m = 32 → 319,008; m = 64 → 635,968 comparisons. The work grows almost exactly as n × m.

3. Inquiry – explanation

- **Why B needs eight times more work than A although n and m are the same:** in A, a mismatch is found at the first character of every alignment (1 comparison per alignment), whereas in B the pattern agrees with the text for m−1 characters before the final character fails, so m comparisons are made per alignment and all the earlier matches are thrown away when the window shifts by one.
- **Characteristics that create repeated comparisons:** a text that contains long runs of the pattern's prefix, a pattern made of repeated characters or with a repeating prefix, and a small alphabet (binary text in test D doubled the work). The mismatch occurring late in the pattern is the key.
- **Input size is not enough:** the best case is Θ(n) comparisons (or even m if a match is at the start) and the worst case is Θ(n·m). Two inputs of the same size n and pattern length m produced 9,993 and 79,944 comparisons. An algorithm must therefore be evaluated with the structure of the data (best, average and worst cases), not only with n. For log screening, signatures made of repetitive characters against repetitive log data can cause the slow behaviour.

Q4 – Closest Pair and the Cost of Examining Every Possibility **6 Marks**

Problem Context – Drone Collision-Risk Monitoring

A logistics company operates delivery drones. At a particular instant, each drone reports its position:

[
D_i=(x_i,y_i)
]

The monitoring system must identify the **two drones closest to one another**.

Your Challenge

Design a **Brute-Force Closest-Pair** solution.

Before implementation, determine:

- Which candidate pairs must be examined
- How duplicate pair comparisons can be avoided
- How many unique pairs exist for (n) drones

Derive the number of candidate pairs mathematically.

Implement your solution and test it using **at least three meaningfully different input sizes**.

Record:

- Number of pairs examined
- Number of distance calculations
- Execution time, where useful

Compare the theoretical growth with experimental behaviour.

Scalability Inquiry

Define what you consider an **acceptable response time** for your experimental environment.

Using your evidence, determine approximately when the brute-force approach begins to become impractical.

Finally, identify **where the computational inefficiency originates** and describe what a redesigned strategy would need to avoid or reduce.

You are not required to implement an advanced Closest-Pair algorithm in this question.

Evidence Expected

Algorithm design, mathematical pair-count derivation, implementation, experimental growth evidence, a defined acceptability criterion and reasoned identification of the limitation of the brute-force strategy.

Answer – Q4: Brute-Force Closest Pair

1. Design

- **Candidate pairs:** every unordered pair of distinct drones $\{D_i, D_j\}$.
- **Avoiding duplicates:** use i from 1 to n−1 and j from i+1 to n only, so the pair (i, j) is examined once and (j, i) and (i, i) are never examined.
- **Distance:** the squared Euclidean distance $(x_i-x_j)^2 + (y_i-y_j)^2$ is compared instead of the distance, because the square root is monotonic; the square root is taken once for the final answer.

```
best = infinity
for i in 0 .. n-2:
    for j in i+1 .. n-1:
        d = (xi-xj)^2 + (yi-yj)^2      # one distance calculation
        if d < best: best = d; pair = (i, j)
return pair, sqrt(best)
```

2. Derivation of the number of candidate pairs

The first drone is paired with n−1 others, the second with n−2 further drones, … and the last-but-one with 1: (n−1) + (n−2) + … + 1 = n(n−1)/2 = C(n,2) = Θ(n²). Equivalently, there are n(n−1) ordered pairs and each unordered pair appears twice, giving n(n−1)/2.

3. Experimental results (random points in a 10,000 × 10,000 area)

n	Pairs examined	Distance calculations	n(n−1)/2	Time (s)
10	45	45	45	0.0
100	4,950	4,950	4,950	0.0007
500	124,750	124,750	124,750	0.0196
1,000	499,500	499,500	499,500	0.0892
2,000	1,999,000	1,999,000	1,999,000	0.3378
4,000	7,998,000	7,998,000	7,998,000	1.2856
8,000	31,996,000	31,996,000	31,996,000	5.5657

The measured number of pairs equals n(n−1)/2 exactly for every size, and one distance calculation is made per pair. When n doubles the pairs increase about four

times (e.g. 1,999,000 → 7,998,000 → 31,996,000) and the time also rises about 4 times (0.34 s → 1.29 s → 5.57 s), so the experiment agrees with the quadratic theory.

4. Scalability inquiry

Acceptable response time: I define it as 1 second for one monitoring cycle in this environment (Python on a laptop-class machine). The measured cost is about 0.16 microseconds per pair. For 1 second this allows about 6 million pairs, i.e. **n ≈ 3,500 drones**; the measured 1.29 s at n = 4,000 already exceeds the limit. For a stricter 100 ms criterion the limit falls to n ≈ 1,100. Brute force therefore becomes impractical at a few thousand drones.

Source of inefficiency: every pair is compared independently, including pairs of drones that are very far apart and can never be the closest pair. The algorithm uses no information about the position of the points and does not reuse the best distance found so far, so the number of distance calculations grows as n^2. A redesigned strategy would need to avoid comparing distant drones, for example by dividing the plane (sorting by coordinate, grid cells or strips) so that each drone is compared only with nearby drones, and by using the current best distance to discard candidates early; the aim is to cut the $n^2/2$ calculations to a number near n log n.

Q5 – Derive a Convex Hull Algorithm from a Geometric Principle 7 Marks

Problem Context – Wildlife Reserve Boundary Monitoring

Monitoring stations are positioned throughout a wildlife reserve. Each station is represented by coordinates:

[
P_i=(x_i,y_i)
]

Conservation officers need to determine the **outer boundary formed by the monitoring stations**.

You are required to develop a **Brute-Force Convex Hull** solution.

Design Starting Point

You are given only the following geometric principle:

> **A line joining two candidate points may form part of the outer boundary if all remaining points lie on an appropriate single side of that line.**

You are **not provided with the complete algorithm**.

Your Challenge

Convert this geometric property into a computational procedure.

Determine:

- How two candidate points define a line
- How the position of another point relative to that line can be determined
- How the remaining points must be examined
- What condition identifies a potential hull edge
- How the final boundary will be constructed

Develop your own pseudocode before implementation.

Test Your Design

Test the algorithm using:

- A point set containing both boundary and interior points
- A second point configuration deliberately designed by you to challenge your algorithm

Visualise the resulting boundary.

Analyse how the computational work grows as the number of points increases.

Correctness Inquiry

Explain:

> **Why does your geometric test provide evidence that an accepted edge belongs to the outer boundary?**

Your explanation should be based on geometric reasoning rather than simply referring to the program output.

Evidence Expected

A student-derived geometric decision rule, pseudocode, implementation, deliberately designed test configurations, computational analysis, boundary visualisation and a reasoned correctness argument.

Answer – Q5: Brute-Force Convex Hull

1. Converting the geometric principle into a procedure

- **Line from two points:** for P_1 (x_1 , y_1) and P_2 (x_2 , y_2) the line is ax + by = c with $a = y_2 - y_1$, $b = x_1 - x_2$, $c = x_1 y_2 - y_1 x_2$.
- **Side of the line:** for any other point P(x, y) compute s = ax + by − c. If s > 0, P is on one side; if s < 0, on the other; if s = 0, P is on the line.
- **Examining remaining points:** compute s for the other n−2 points.
- **Condition for a hull edge:** all values of s are ≥ 0 or all are ≤ 0 (no point on both sides). Points with s = 0 are allowed only if they lie between P_1 and P_2 on the segment; otherwise the true edge is longer and this pair is rejected.
- **Constructing the boundary:** collect all accepted segments, then follow them from one end point to the next (each hull vertex has exactly two accepted edges) to obtain the polygon in order. Duplicate points are removed first.

2. Pseudocode

```
remove duplicate points; if n < 3 return the points
edges = []
for i in 0 .. n-2:
  for j in i+1 .. n-1:
    a = yj-yi; b = xi-xj; c = xi*yj - yi*xj
    pos = neg = false; ok = true
    for k != i, j:
      s = a*xk + b*yk - c
      if s > 0: pos = true
      if s < 0: neg = true
      if s == 0 and Pk is outside segment PiPj: ok = false
      if pos and neg: ok = false; break
    if ok: edges.add((Pi, Pj))
```

```
link the edges end-to-end to give the ordered boundary
```

3. Tests

- **Configuration 1 (boundary + interior):** 12 points, 7 of them interior. Hull found: (2,1), (9,2), (11,7), (6,12), (1,8) – 5 vertices; 331 orientation tests.
- **Configuration 2 (designed to challenge the algorithm):** a square of side 8 with extra points lying on its edges (collinear boundary points such as (4,0), (2,0), (8,4)), three interior points on a diagonal, and a duplicate point. 14 given points, 13 distinct. The algorithm returned only the four corners (0,0), (8,0), (8,8), (0,8) – the collinear points on the edges were correctly not reported as vertices, the duplicate was removed and the interior points were rejected; 383 tests.
- Further degenerate tests: 8 points all on one line → hull is the two end points (0,1) and (7,15) (1 edge); 12 points on a circle → all 12 are hull vertices (12 edges).

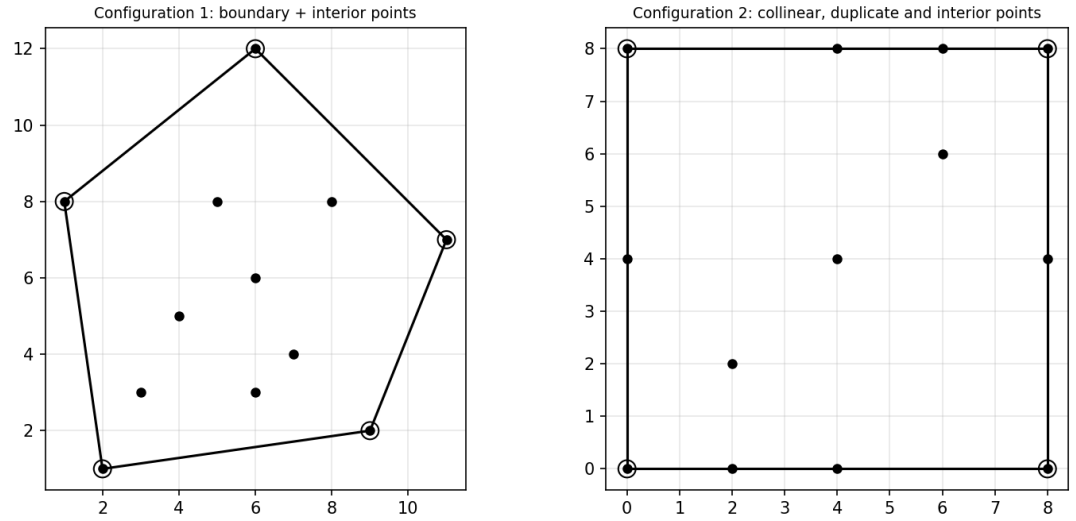


Figure: hull boundary (line) and hull vertices (circled) for the two configurations.

4. Growth of computational work

n	Orientation tests performed	Maximum n(n−1)(n−2)/2	Hull vertices	Time (s)
10	218	360	6	0.0001
20	1,323	3,420	8	0.0004
40	8,887	29,640	11	0.0021
80	61,283	246,480	10	0.0133
160	452,480	2,009,760	14	0.0921
320	3,417,357	16,230,720	20	0.7042

There are $n(n-1)/2$ candidate pairs and each is checked against up to $n-2$ points, so the work is $\Theta(n^3)$ in the worst case. Early exit (stopping when points are found on both sides) reduces the count, but the growth is still cubic: when n doubles the tests increase by about 7.4–7.6 times (61,283 → 452,480 → 3,417,357), close to the factor of 8 expected for n^3.

5. Correctness argument

Let L be the line through two input points P and Q and assume every other point lies on one closed side of L (or on L between P and Q). Then L is a supporting line: the whole point set lies in one half-plane bounded by L. The convex hull is the smallest convex set containing all points, so it also lies in that half-plane. Because P and Q belong to the hull and the hull is convex, the segment PQ lies in the hull and also on L. A point of the segment cannot be an interior point of the hull, since every neighbourhood of it contains points on the other side of L that are outside the hull. Hence PQ lies on the boundary, and an accepted edge really belongs to the outer boundary.

Conversely, every edge of the hull lies on a supporting line, so all other points are on one side of it and the test accepts it; no true edge is missed. If some point lay on each side of L, then L would pass through the inside of the set and PQ would cut through the hull, so rejecting it is also correct. The extra rule for collinear points prevents the reporting of short sub-segments of a longer edge.

Q6 – Exhaustive Search and Combinatorial Growth **8 Marks**

Problem Context – Humanitarian Airlift Planning

A helicopter carrying emergency supplies has a strict payload limit.

Each available package contains:

Package ID | Weight | Humanitarian Value

The objective is to select a combination of packages that:

- Does not exceed the payload limit, and
- Maximises the total humanitarian value delivered.

Your Challenge

Design an **Exhaustive-Search strategy**.

Determine:

- How candidate package combinations will be represented
- How candidate solutions will be generated
- How feasibility will be checked
- How feasible solutions will be evaluated
- How the best solution will be retained

Do not begin from supplied pseudocode.

Correctness Check

Create a small instance for which the possible combinations can be independently verified.

Use this to establish whether your algorithm identifies the correct optimum.

Growth Investigation

Investigate increasing problem sizes.

Before experimentation:

- Derive/predict how the number of candidate combinations grows with (n).
- Predict what will happen as additional packages are introduced.

Use **at least three meaningful problem sizes** to validate your prediction experimentally.

Inquiry

Explain:

Why can adding what appears to be a relatively small number of additional packages produce a dramatic increase in computational effort?

Your explanation must relate to the growth of the **solution space**.

Deployment Decision

Assume that the airlift planner must produce a loading decision within a fixed response time selected and justified by you.

Using your experimental evidence, estimate the largest problem size for which you would be willing to deploy your exhaustive-search solution.

Evidence Expected

Student-designed exhaustive enumeration, correctness verification, mathematical prediction of solution-space growth, experimental validation, explanation of combinatorial growth and an evidence-based deployment boundary.

Answer – Q6: Exhaustive Search for the Airlift Load

1. Design of the exhaustive search

- **Representation:** with n packages, each candidate is an n-bit binary string (bit i = 1 means package i is loaded), equivalent to an integer mask from 0 to $2^n - 1$.
- **Generation:** loop over all masks 0 … $2^n - 1$, which produces every subset exactly once.
- **Feasibility check:** add up the weights of the selected packages; the subset is feasible only if the total weight ≤ payload limit.
- **Evaluation:** for a feasible subset add up the humanitarian values.
- **Retention of best:** keep the highest value found and the corresponding mask; replace it only if a feasible candidate has a higher value.

```
best = -1; best_mask = 0
for mask in 0 .. 2^n - 1:
    W = 0; V = 0
    for i in 0 .. n-1:
        if bit i of mask is 1: W += w[i]; V += v[i]
    if W <= capacity and V > best:
        best = V; best_mask = mask
return best, best_mask
```

2. Correctness check on a small instance (payload limit 10 kg)

Packages: A (5 kg, value 10), B (4 kg, value 40), C (6 kg, value 30), D (3 kg, value 50). There are $2^4 = 16$ subsets, all listed below and easily checked by hand.

Subset	Weight	Value	Feasible?
{ } (empty)	0	0	Yes
{A}	5	10	Yes
{B}	4	40	Yes
{A,B}	9	50	Yes
{C}	6	30	Yes
{A,C}	11	40	No (over 10)
{B,C}	10	70	Yes
{A,B,C}	15	80	No (over 10)
{D}	3	50	Yes
{A,D}	8	60	Yes
{B,D}	7	90	Yes

{A,B,D}	12	100	No (over 10)
{C,D}	9	80	Yes
{A,C,D}	14	90	No (over 10)
{B,C,D}	13	120	No (over 10)
{A,B,C,D}	18	130	No (over 10)

The best feasible subset is **{B, D}** with weight 7 kg and value **90**; the next best are {C, D} = 80 and {B, C} = 70. The program returned value 90 with mask 1010 (B and D), so it identifies the correct optimum.

3. Growth investigation – prediction

Each package is either in or out of the load, so the number of candidates is $2 \times 2 \times \ldots \times 2 = 2^n$. Adding one package doubles the candidates; adding k packages multiplies them by 2^k. Since each candidate is evaluated in O(n), the total work is about $n \cdot 2^n$. Predicted: n = 10 → 1,024 candidates; n = 20 → 1,048,576; n = 22 → 4,194,304; and time should rise about four times for every 2 extra packages.

4. Experimental validation (same packages, payload = 40% of total weight)

n	Candidates examined	2^n	Feasible candidates	Best value	Time (s)	Bit tests ($n \cdot 2^n$)
10	1,024	1,024	292	339	0.001	10,240
14	16,384	16,384	3,965	482	0.021	229,376
18	262,144	262,144	56,989	700	0.428	4,718,592
20	1,048,576	1,048,576	218,630	777	2.019	20,971,520
22	4,194,304	4,194,304	833,691	877	9.477	92,274,688

The number of candidates examined equals 2^n exactly for every size, confirming the prediction. The time increased about 4.7 times for every two extra packages from n = 18 to 22 (0.43 s → 2.02 s → 9.48 s), a little more than the predicted 4 times because the work per candidate also grows with n.

5. Inquiry – why a small increase in packages gives a dramatic increase in effort

The solution space is exponential: every new package doubles the number of subsets, because each existing subset can be taken with or without the new package. Increasing n from 20 to 30 is only 50% more packages, yet it makes the space $2^{10} = 1{,}024$ times larger (over one billion subsets), whereas a polynomial such

as n^2 would grow only from 400 to 900. The brute-force algorithm cannot use any structure to skip subsets, so its running time follows the size of the whole solution space and adding even a few packages results in a very large increase in effort.

6. Deployment decision

Response-time limit chosen: 5 seconds, since a loading plan must be ready while the aircraft is being prepared and a planner should not wait longer for an interactive decision. From the experiments, n = 20 takes 2.0 s and n = 22 takes 9.5 s, and each extra package roughly doubles the time (n = 21 would take about 4–5 s, which is too close to the limit). I would therefore deploy the exhaustive-search solution for **at most 20 packages**, with n = 21 as the absolute upper limit. For larger loads a smarter method is needed (this is the subject of Q7).

DAY-4 DESIGN CHALLENGE

Q7 – Diagnose, Redesign and Validate 13 Marks

Review the evidence you generated while working on Q1–Q6.

Select **ONE solution** that demonstrates a significant computational limitation as its problem size or operating requirement increases.

You are **not permitted simply to replace the solution with a more advanced textbook algorithm and state that its complexity is better**.

Your task is to understand the computational problem first and then develop an appropriate redesign.

A. Diagnose the Computational Bottleneck 2 Marks

Using your own mathematical and/or experimental evidence, identify:

- Where most computational effort is being spent
- Which operation grows significantly as the input increases
- Whether work is being unnecessarily repeated
- Which characteristic of the original algorithm causes the limitation

Evidence Expected

A precise diagnosis supported by evidence from your original implementation and analysis.

Answer – Q7A: Diagnose the Computational Bottleneck

Solution chosen for redesign: Q6 (Exhaustive Search for the airlift load). It has the most severe limitation: the deployment boundary was only about 20 packages.

n	Candidates 2^n	Feasible candidates	Feasible share	Infeasible (wasted) candidates	Time (s)
10	1,024	292	28.5%	732	0.001
14	16,384	3,965	24.2%	12,419	0.021
18	262,144	56,989	21.7%	205,155	0.428
20	1,048,576	218,630	20.9%	829,946	2.019
22	4,194,304	833,691	19.9%	3,360,613	9.477

- **Where most effort is spent:** in generating and evaluating subsets. At n = 22 the program examined 4,194,304 candidates and used 92,274,688 bit tests ($n \cdot 2^n$), which took 9.48 s.
- **Which operation grows:** the number of candidates, which doubles for each added package (2^n), multiplied by the O(n) cost of summing each candidate.
- **Unnecessary repeated work:** (a) the weight and value of every subset are recomputed from the beginning, although subsets share long common prefixes; a decision tree would need only $2^{n+1} - 1 = 8{,}388{,}607$ incremental steps at n = 22, about 11 times fewer than the 92 million bit tests; (b) about 80% of the candidates (3,360,613 at n = 22) are infeasible and the share rises as n grows (71.5% at n = 10 to 80.1% at n = 22), yet each is completely generated and weighed; (c) when a partial selection already exceeds the payload, all its supersets are examined although they are certainly infeasible.
- **Cause in the algorithm:** it enumerates blindly and independently – it does not use the capacity limit to cut the search, does not use the best value found so far, and does not reuse the sums of shared partial selections.

B. Define the Design Objective **2 Marks**

Before proposing a new solution, clearly state what your redesign must achieve.

For example:

- Reduce unnecessary comparisons
- Avoid examining every candidate
- Exploit existing ordering
- Reuse previously computed information
- Reduce the number of candidate solutions
- Reduce repeated processing

Specify a **measurable objective** wherever possible.

Evidence Expected

A clearly stated redesign objective derived directly from the identified computational limitation.

Answer – Q7B: Design Objective

The redesigned solution must:

- **Remain exact:** return the same optimal value as the exhaustive search on every test instance (no heuristic answers).
- **Avoid examining every candidate:** examine fewer than 1% of the 2^n candidates at n = 22 (below 42,000 instead of 4,194,304) and reduce the effort by at least 100 times.
- **Never fully evaluate infeasible selections:** stop extending a partial selection when its weight exceeds the payload.
- **Reuse previously computed information:** maintain the running weight and value so that adding a package costs O(1) instead of O(n).
- **Extend the deployable size:** solve instances with $n \geq 40$ within the 5-second limit used in Q6 (the original method stopped at n = 20).

C. Design the Revised Computational Strategy **4 Marks**

Develop a revised solution.

Explain:

- What you have changed
- What information or property your new design exploits
- Which operations are avoided or reduced
- Whether preprocessing is introduced
- Whether additional memory is required
- Any new trade-offs created by the redesign

Provide appropriate pseudocode or a design representation.

Your marks will be based on the **quality of the algorithmic reasoning**, not merely on whether you name a known algorithm.

Evidence Expected

A coherent redesigned computational strategy directly addressing the diagnosed bottleneck, supported by appropriate design representation and technical justification.

Answer – Q7C: Revised Computational Strategy

What is changed

The flat enumeration of masks is replaced with a **decision-tree search with pruning (branch and bound)**. Packages are considered one by one; at each level there are two choices, include or exclude the package. Whole branches are abandoned when they cannot lead to a better feasible solution.

Properties exploited and operations avoided

- **Capacity constraint (feasibility pruning):** if adding a package exceeds the payload, the include branch and all 2^(remaining) subsets below it are skipped.
- **Value density ordering (preprocessing):** packages are sorted by value/weight in decreasing order (O(n log n)), so good solutions are found early, and the best value rises quickly.
- **Optimistic bound:** for a partial selection, an upper bound is the current value plus the value obtained by filling the remaining capacity with the remaining packages in density order, allowing the last one to be taken fractionally. No integral completion can exceed this bound, because the fractional version

relaxes the problem. If bound ≤ best value found so far, the entire subtree is pruned.

- **Incremental sums:** the running weight and value are passed down the tree, which removes the factor n of re-summing each candidate.

Pseudocode

```
sort packages by v/w descending
best = 0
search(i, W, V):
    best = max(best, V)
    if i == n: return
    if bound(i, W, V) <= best: return        # prune: cannot win
    if W + w[i] <= capacity:                 # feasibility prune
        search(i+1, W + w[i], V + v[i])      # include package i
    search(i+1, W, V)                        # exclude package i

bound(i, W, V):                              # fractional fill
    room = capacity - W; b = V
    while i < n and w[i] <= room:
        room -= w[i]; b += v[i]; i += 1
    if i < n: b += v[i] * room / w[i]
    return b
```

Trade-offs

- **Preprocessing:** the sort, O(n log n).
- **Memory:** O(n) for the recursion stack and the sorted arrays; the original needed only O(1) extra memory.
- **Cost per node:** the bound computation can cost up to O(n) per node.
- **Predictability:** the running time now depends on the data. The worst case is still exponential, for instance when values are almost proportional to weights and the bound prunes poorly.
- The code is more complex and harder to verify than the exhaustive search, which is why the exhaustive version is kept as a reference for testing.

D. Predict Before Implementing **1 Mark**

Before testing the redesigned solution:

- Predict how its behaviour should differ from the original.
- Identify the performance measure in which you expect improvement.

Evidence Expected

A clear, testable prediction made before experimental validation.

Answer – Q7D: Prediction Before Implementing

- **Predicted behaviour:** the redesigned algorithm will return the same optimal value as the original but will examine a tiny fraction of the 2^n candidates; the number of nodes visited will grow slowly with n for ordinary (random) instances instead of doubling for each extra package.
- **Performance measure expected to improve:** the number of candidate solutions/nodes examined (primary), and then execution time.
- **Expected exception:** on instances in which value is almost proportional to weight (strongly correlated), the bound will be weak and pruning less effective, so I expect a smaller improvement and faster growth of nodes there.
- **Quantitative expectation:** at n = 22, fewer than 42,000 nodes (1% of 4,194,304).

E. Implement and Validate the Redesign

Implement a prototype of the redesigned strategy wherever feasible.

Using the **same or equivalent test conditions**, compare:

Original Solution → Redesigned Solution

Use appropriate measures such as:

- Comparisons
- Key operations
- Candidate solutions examined
- Distance calculations
- Character comparisons
- Execution time, where relevant

Evidence Expected

Comparable experimental evidence showing the behaviour of the original and redesigned solutions

Answer – Q7E: Implementation and Validation

The prototype was run on the same instances as in Q6 (same packages and the same payload of 40% of total weight). Primary evidence is the number of candidate nodes examined; time is additional evidence.

n	Original: candidates	Original time (s)	Redesign: nodes	Redesign time (s)	Optimum original / redesign
10	1,024	0.001	41	6e-05	339 / 339
14	16,384	0.021	40	7e-05	482 / 482
18	262,144	0.428	82	0.0001	700 / 700
20	1,048,576	2.019	70	8e-05	777 / 777
22	4,194,304	9.477	48	7e-05	877 / 877

The optimum agreed on all five sizes. Correctness was also tested on 200 random instances with 3 to 12 packages: the redesign and the exhaustive search returned the same optimum in all 200 cases (0 mismatches).

Larger instances (the original method cannot be run for these sizes):

n	Optimal value	Nodes examined	Time (s)	Candidates in exhaustive search (2^n)
30	1171	51	0.0001	2^30
40	1531	244	0.0002	2^40
60	2452	137	0.0002	2^60
100	3537	213	0.0005	2^100

Harder test – strongly correlated instances (value = weight + 10):

n	Nodes examined (redesign)	Time (s)	Exhaustive candidates 2^n	Reduction
20	1,561	0.0008	1,048,576	672 times
30	9,543	0.0041	1,073,741,824	112,516 times
40	21,564	0.009	1,099,511,627,776	50,988,297 times

F. Defend the Final Design **2 Marks**

Evaluate whether the redesign achieved its intended objective.

Discuss:

- Whether the prediction was supported
- Magnitude of improvement, if any
- New costs or trade-offs
- Situations in which the original solution may still be preferable
- Whether the redesigned solution sufficiently addresses the identified limitation

A redesign that does **not** outperform the original will not automatically be considered unsuccessful if you correctly analyse and explain why the design hypothesis did not produce the expected result.

Evidence Expected

A critical, evidence-based evaluation of the redesigned solution and a defensible final engineering conclusion.

Answer – Q7F: Defence of the Final Design

- **Was the prediction supported?** Yes. The redesign gave the same optimum in every test and examined far fewer candidates. At n = 22 it examined 48 nodes instead of 4,194,304 (about 87,000 times fewer, well inside the objective of 1%). Time fell from 9.48 s to under 0.1 ms. The predicted exception was also seen: on strongly correlated data the nodes grew from 1,561 (n = 20) to 9,543 (n = 30) to 21,564 (n = 40), much more than on random data, which varied between roughly 40 and 250 nodes.
- **Magnitude of improvement:** about 87,000 times fewer candidates on random data at n = 22, and 672 times fewer (1,561 against 1,048,576) on the difficult correlated data at n = 20, growing to about 112,000 times fewer at n = 30. The objective of solving n ≥ 40 within 5 seconds was achieved: n = 100 random packages took 0.0005 s and n = 40 correlated packages 0.009 s.
- **New costs and trade-offs:** sorting preprocessing, O(n) extra memory, an O(n) bound computation at each node, more complex code and more difficult verification. Most importantly, the running time is no longer predictable from n alone and the worst case is still exponential; strongly correlated instances show it can grow quickly (about 2.3 times from n = 30 to n = 40).
- **When the original is still preferable:** for very small instances (about n ≤ 12) where it is instant and its simplicity is a benefit, as an independent reference to test the optimised program, and when a guaranteed bounded amount of work regardless of data is required for a very simple, easily certified implementation.

- **Does it sufficiently address the limitation?** Yes for realistic airlift planning: the deployment boundary rose from about 20 packages to at least 100 packages on random data and at least 40 on the hard correlated data, while remaining exact. The limitation (blind enumeration of 2^n candidates) has been addressed by feasibility pruning, bounding and incremental evaluation; however, it has not been removed in the worst case, so for very large adversarial instances a different approach (for example dynamic programming on the capacity) would be the next step.

MARK DISTRIBUTION

Question	Assessed Area	Marks
Q1	Selection, Bubble & Insertion Sort	**6**
Q2	Sequential Search	**4**
Q3	Brute-Force String Matching	**6**
Q4	Brute-Force Closest Pair	**6**
Q5	Brute-Force Convex Hull	**7**
Q6	Exhaustive Search	**8**
Q7	**Day-4 Design Challenge – Diagnose, Redesign & Validate**	**13**
	TOTAL	**50**

Submission Requirements

Submit:

- Your assignment report containing the required evidence for Q1–Q7
- Source code for all implemented solutions
- Raw experimental data used to produce your results
- Tables/graphs used in your analysis, where applicable

Operation counts, comparison counts or other algorithm-specific measures should be treated as **primary experimental evidence**. Execution time may be reported as additional evidence but should not be used as the sole basis for your conclusions.

Assessment Verification

Students may be asked to explain any part of their:

- Algorithmic design
- Source code
- Test-data design
- Experimental results
- Predictions
- Analysis
- Design Challenge

during a brief verification discussion/viva.

The student should therefore be able to **explain and defend the evidence submitted as their own work**.

Overall Assessment Expectation

This assignment does not primarily assess your ability to reproduce textbook algorithms.

It assesses your ability to:

Predict → Design → Implement → Measure → Analyse → Evaluate → Diagnose → Redesign → Validate

Your conclusions and design decisions must be supported by **your own mathematical reasoning and experimental evidence**.